

Task 1:

```
% conda info
```

```

    active environment : base
    active env location : /opt/anaconda3
        shell level : 1
        user config file : /Users/lingyixu/.condarc
    populated config files : /Users/lingyixu/.condarc
        conda version : 24.7.1
    conda-build version : 24.5.1
        python version : 3.12.4.final.0
            solver : libmamba (default)
    virtual packages : __archspec=1=m1
                      __conda=24.7.1=0
                      __osx=14.6.1=0
                      __unix=0=0
    base environment : /opt/anaconda3 (writable)
    conda av data dir : /opt/anaconda3/etc/conda
    conda av metadata url : None
        channel URLs :
https://repo.anaconda.com/pkgs/main/osx-arm64
                      https://repo.anaconda.com/pkgs/main/no
arch
                      https://repo.anaconda.com/pkgs/r/osx-a
rm64
                      https://repo.anaconda.com/pkgs/r/noarc
h
        package cache : /opt/anaconda3/pkgs
                      /Users/lingyixu/.conda/pkgs
    envs directories : /opt/anaconda3/envs
                      /Users/lingyixu/.conda/envs
        platform : osx-arm64
        user-agent : conda/24.7.1 requests/2.32.2
CPython/3.12.4 Darwin/23.6.0 OSX/14.6.1 solver/libmamba
conda-libmamba-solver/24.1.0 libmambapy/1.5.8 aau/0.4.4 c/. s/.
e/.

        UID:GID : 501:20
        netrc file : None
    offline mode : False

```

Task 2:

Code in IPython	Results
<code>a = np.array([[1, 2, 3], [4, 5, 6]])</code> <code>np.ndim(a)</code>	2
<code>np.size(a)</code>	6
<code>np.shape(a)</code>	(2, 3)
<code>n = 2</code> <code>a.shape[n-1]</code>	3
<code>np.array([[1., 2., 3.], [4., 5., 6.]])</code>	array([[1., 2., 3.], [4., 5., 6.]])
<code>a = np.array([[1, 2], [3, 4]])</code> <code>b = np.array([[5, 6], [7, 8]])</code> <code>c = np.array([[9, 10], [11, 12]])</code> <code>d = np.array([[13, 14], [15, 16]])</code> <code>np.block([[a, b], [c, d]])</code>	array([[1, 2, 5, 6], [3, 4, 7, 8], [9, 10, 13, 14], [11, 12, 15, 16]])
<code>a = np.array([1, 2, 3, 4, 5])</code> <code>a[-1]</code>	5
<code>a = np.array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])</code> <code>a[1, 4]</code>	10
<code>a[1, :]</code>	array([6, 7, 8, 9, 10])
<code>a[:5, :]</code>	array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
<code>a[-5:]</code>	array([[1, 2, 3, 4, 5], [6, 7, 8, 9, 10]])
<code>a = np.array([[1, 2, 3, 4, 5, 56, 57, 58, 59], [6, 7, 8, 9, 10, 66, 67, 68, 69], [11, 12, 13, 14, 15, 76, 77, 78, 79], [16, 17, 18, 19, 20, 86, 87, 88, 89], [21, 22, 23, 24, 25, 96, 97, 98, 99]])</code> <code>a[0:3, 4:9]</code>	array([[5, 56, 57, 58, 59], [10, 66, 67, 68, 69], [15, 76, 77, 78, 79]])
<code>a[np.ix_([1, 3, 4], [0, 2])]</code>	array([[6, 8], [16, 18], [21, 23]])
<code>a[2:21:2, :]</code>	array([[11, 12, 13, 14, 15, 76, 77, 78, 79], [21, 22, 23, 24, 25, 96, 97, 98, 99]])
<code>a[:, :2, :]</code>	array([[1, 2, 3, 4, 5, 56, 57, 58, 59],

	<pre> [11, 12, 13, 14, 15, 76, 77, 78, 79], [21, 22, 23, 24, 25, 96, 97, 98, 99]]) </pre>
<code>a[::-1,:]</code>	<pre> array([[21, 22, 23, 24, 25, 96, 97, 98, 99], [16, 17, 18, 19, 20, 86, 87, 88, 89], [11, 12, 13, 14, 15, 76, 77, 78, 79], [6, 7, 8, 9, 10, 66, 67, 68, 69], [1, 2, 3, 4, 5, 56, 57, 58, 59]]) </pre>
<code>a[np.r_[:len(a),0]]</code>	<pre> array([[1, 2, 3, 4, 5, 56, 57, 58, 59], [6, 7, 8, 9, 10, 66, 67, 68, 69], [11, 12, 13, 14, 15, 76, 77, 78, 79], [16, 17, 18, 19, 20, 86, 87, 88, 89], [21, 22, 23, 24, 25, 96, 97, 98, 99], [1, 2, 3, 4, 5, 56, 57, 58, 59]]) </pre>
<code>a.transpose()</code>	<pre> array([[1, 6, 11, 16, 21], [2, 7, 12, 17, 22], [3, 8, 13, 18, 23], [4, 9, 14, 19, 24], [5, 10, 15, 20, 25], [56, 66, 76, 86, 96], [57, 67, 77, 87, 97], [58, 68, 78, 88, 98], [59, 69, 79, 89, 99]]) </pre>
<code>a.conj().transpose()</code>	<pre> array([[1, 6, 11, 16, 21], [2, 7, 12, 17, 22], [3, 8, 13, 18, 23], [4, 9, 14, 19, 24], [5, 10, 15, 20, 25], [56, 66, 76, 86, 96], [57, 67, 77, 87, 97], [58, 68, 78, 88, 98], [59, 69, 79, 89, 99]]) </pre>

<pre>a = np.array([[1, 2], [3, 4]]) b = np.array([[5, 6], [7, 8]]) a @ b</pre>	<pre>array([[19, 22], [43, 50]])</pre>
<pre>a * b</pre>	<pre>array([[5, 12], [21, 32]])</pre>
<pre>a/b</pre>	<pre>array([[0.2 , 0.33333333], [0.42857143, 0.5]])</pre>
<pre>a**3</pre>	<pre>array([[1, 8], [27, 64]])</pre>
<pre>(a > 0.5)</pre>	<pre>array([[True, True], [True, True]])</pre>
<pre>np.nonzero(a > 0.5)</pre>	<pre>(array([0, 0, 1, 1]), array([0, 1, 0, 1]))</pre>
<pre>v = np.array([0.2, 0.8]) a[:, np.nonzero(v > 0.5)[0]]</pre>	<pre>array([[2], [4]])</pre>
<pre>a[:, v.T > 0.5]</pre>	<pre>array([[2], [4]])</pre>
<pre>a = np.array([[0.2, 0.3, 0.4, 0.5], [1.2, 1.3, 1.4, 1.5], [0.6, 1.1, 0.1, 0.7]]) a[a < 0.5]=0 a</pre>	<pre>array([[0. , 0. , 0. , 0.5], [1.2, 1.3, 1.4, 1.5], [0.6, 1.1, 0. , 0.7]])</pre>
<pre>a * (a > 1)</pre>	<pre>array([[0. , 0. , 0. , 0.], [1.2, 1.3, 1.4, 1.5], [0. , 1.1, 0. , 0.]])</pre>
<pre>a[:] = 3 a</pre>	<pre>array([[3., 3., 3., 3.], [3., 3., 3., 3.], [3., 3., 3., 3.]])</pre>
<pre>b = a.copy() b</pre>	<pre>array([[3., 3., 3., 3.], [3., 3., 3., 3.], [3., 3., 3., 3.]])</pre>
<pre>a = np.array([[0.2, 0.3, 0.4, 0.5], [1.2, 1.3, 1.4, 1.5], [0.6, 1.1, 0.1, 0.7]]) b = a[1, :].copy() b</pre>	<pre>array([1.2, 1.3, 1.4, 1.5])</pre>
<pre>a.flatten()</pre>	<pre>array([0.2, 0.3, 0.4, 0.5, 1.2, 1.3, 1.4, 1.5, 0.6, 1.1, 0.1, 0.7])</pre>
<pre>np.arange(1., 11.)</pre>	<pre>array([1., 2., 3., 4., 5., 6., 7., 8., 9., 10., 11.])</pre>

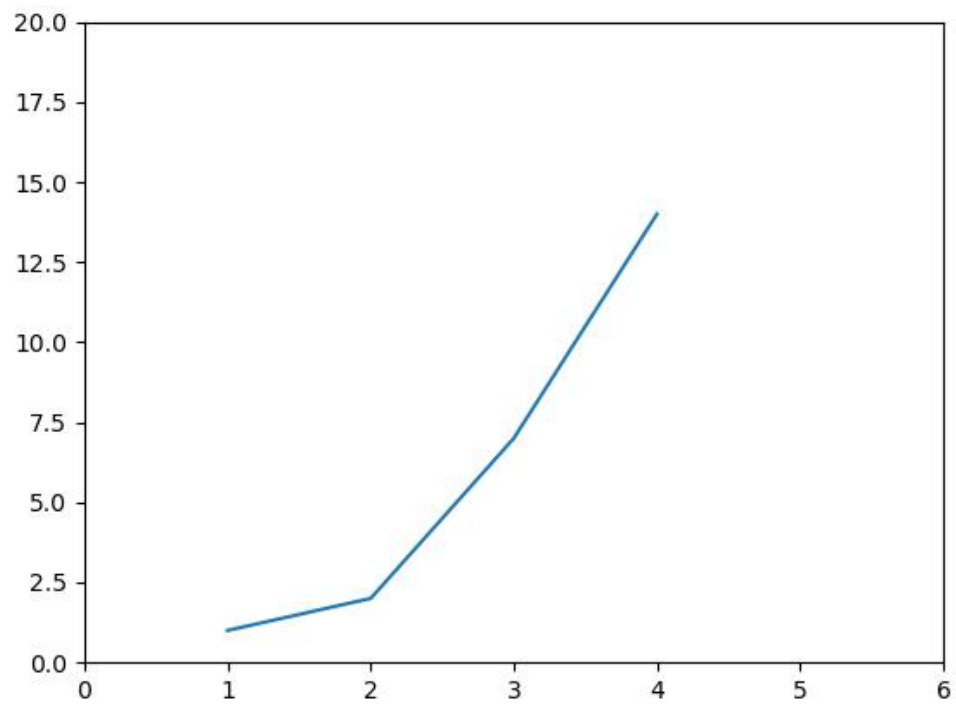
	7., 8., 9., 10.]])
<code>np.arange(10.)</code>	<code>array([0., 1., 2., 3., 4., 5., 6., 7., 8., 9.]</code>
<code>np.arange(1.,11.)[:, np.newaxis]</code>	<code>array([[1., [2., [3., [4., [5., [6., [7., [8., [9., [10.]])</code>
<code>np.zeros((3, 4))</code>	<code>array([[0., 0., 0., 0.], [0., 0., 0., 0.], [0., 0., 0., 0.]])</code>
<code>np.zeros((3, 4, 5))</code>	<code>array([[[0., 0., 0., 0., 0.], [0., 0., 0., 0., 0.], [0., 0., 0., 0., 0.], [0., 0., 0., 0., 0.]], [[[0., 0., 0., 0., 0.], [0., 0., 0., 0., 0.], [0., 0., 0., 0., 0.], [0., 0., 0., 0., 0.]], [[[0., 0., 0., 0., 0.], [0., 0., 0., 0., 0.], [0., 0., 0., 0., 0.], [0., 0., 0., 0., 0.]]])</code>
<code>np.ones((3, 4))</code>	<code>array([[1., 1., 1., 1.], [1., 1., 1., 1.], [1., 1., 1., 1.]])</code>
<code>np.eye(3)</code>	<code>array([[1., 0., 0.], [0., 1., 0.], [0., 0., 1.]])</code>
<code>np.diag(a)</code>	<code>array([0.2, 1.3, 0.1])</code>
<code>from numpy.random import default_rng rng = default_rng(42) rng.random((3, 4))</code>	<code>array([[0.77395605, 0.43887844, 0.85859792, 0.69736803], [0.09417735, 0.97562235, 0.7611397 , 0.78606431], [0.12811363, 0.45038594, 0.37079802, 0.92676499]])</code>
<code>np.linspace(1,3,4)</code>	<code>array([1. , 1.66666667,</code>

	2.33333333, 3.])
<code>np.mgrid[0:9.,0:6.]</code>	<pre>array([[0., 0., 0., 0., 0., 0.], [1., 1., 1., 1., 1., 1.], [2., 2., 2., 2., 2., 2.], [3., 3., 3., 3., 3., 3.], [4., 4., 4., 4., 4., 4.], [5., 5., 5., 5., 5., 5.], [6., 6., 6., 6., 6., 6.], [7., 7., 7., 7., 7., 7.], [8., 8., 8., 8., 8., 8.]], [[0., 1., 2., 3., 4., 5.], [0., 1., 2., 3., 4., 5.], [0., 1., 2., 3., 4., 5.], [0., 1., 2., 3., 4., 5.], [0., 1., 2., 3., 4., 5.], [0., 1., 2., 3., 4., 5.], [0., 1., 2., 3., 4., 5.], [0., 1., 2., 3., 4., 5.], [0., 1., 2., 3., 4., 5.]])</pre>
<code>np.ix_(np.r_[0:9.],np.r_[0:6.]</code>	<pre>(array([[0.], [1.], [2.], [3.], [4.], [5.], [6.], [7.], [8.]]) , array([[0., 1., 2., 3., 4., 5.]])</pre>
<code>np.meshgrid([1,2,4],[2,4,5])</code>	<pre>[array([[1, 2, 4], [1, 2, 4], [1, 2, 4]]), array([[2, 2, 2], [4, 4, 4], [5, 5, 5]])]</pre>
<code>np.ix_([1,2,4],[2,4,5])</code>	<pre>(array([[1], [2], [4]]), array([[2, 4, 5]]))</pre>
<code>a = np.array([[1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]])</code>	<pre>array([[1, 2, 3, 4, 1, 2, 3, 4], [5, 6, 7, 8, 5, 6, 7, 8], [9, 10, 11, 12, 9, 10, 11, 12]])</pre>

m = 1 n = 2 np.tile(a, (m, n))	
a=np.array([[1, 2, 3, 4], [5, 6, 7, 8]]) b=np.array([[11, 12, 13, 14], [15, 16, 17, 18]]) np.concatenate((a,b),1)	array([[1, 2, 3, 4, 11, 12, 13, 14], [5, 6, 7, 8, 15, 16, 17, 18]])
np.concatenate((a,b))	array([[1, 2, 3, 4], [5, 6, 7, 8], [11, 12, 13, 14], [15, 16, 17, 18]])
a.max()	8
a.max(0)	array([5, 6, 7, 8])
a.max(1)	array([4, 8])
np.maximum(a, b)	array([[11, 12, 13, 14], [15, 16, 17, 18]])
np.linalg.norm(a)	14.2828568570857
np.logical_and(a,b)	array([[True, True, True, True], [True, True, True, True]])
np.logical_or(a,b)	array([[True, True, True, True], [True, True, True, True]])
a & b	array([[1, 0, 1, 4], [5, 0, 1, 0]])
a b	array([[11, 14, 15, 14], [15, 22, 23, 26]])
c = np.array([[9, 10], [11, 12]]) np.linalg.inv(c)	array([[-6. , 5.], [5.5, -4.5]])
np.linalg.pinv(c)	array([[-6. , 5.], [5.5, -4.5]])
np.linalg.matrix_rank(c)	2
np.linalg.solve(c, b)	array([[9., 8., 7., 6.], [-7., -6., -5., -4.]])
np.linalg.lstsq(a, b)	(array([[-2.3, -2.6, -2.9, -3.2], [-0.6, -0.7, -0.8, -0.9], [1.1, 1.2, 1.3, 1.4], [2.8, 3.1, 3.4, 3.7]]), array([], dtype=float64), 2, array([14.22740741, 1.25732984]))
U, S, Vh = np.linalg.svd(a); V = Vh.T	In [94]: U Out[94]: array([[-0.37616823, -0.92655138],

	<pre>[-0.92655138, 0.37616823]]) In [95]: S Out[95]: array([14.22740741, 1.25732984]) In [96]: Vh Out[96]: array([[-0.35206169, -0.44362578, -0.53518987, -0.62675396], [0.75898127, 0.3212416 , -0.11649807, -0.55423774], [-0.40008743, 0.25463292, 0. 69099646, -0.54554195], [-0.37407225, 0.79697056, -0.47172438, 0.04882607]])</pre>
<pre>d = np.array([[5, 4], [3, 4]]) np.linalg.cholesky(d)</pre>	<pre>array([[2.23606798, 0.], [1.34164079, 1.4832397]])</pre>
<pre>D,V = np.linalg.eig(d)</pre>	<pre>In [106]: D Out[106]: array([8., 1.]) In [107]: V Out[107]: array([[0.8 , -0.70710678], [0.6 , 0.70710678]])</pre>
<pre>D,V = np.linalg.eig(a, b)</pre>	<pre>TypeError: eig() takes 1 positional argument but 2 were given Do not know why.</pre>
<pre>D,V = eigs(a, k=3)</pre>	<pre>NameError: name 'eigs' is not defined; module 'numpy' has no attribute 'eigs'; eig() got an unexpected keyword argument 'k'; Do not know how to deal with it.</pre>
<pre>Q,R = np.linalg.qr(a) Q, R</pre>	<pre>(array([[-0.19611614, -0.98058068], [-0.98058068, 0.19611614]]), array([[-5.09901951, -6.27571632, -7.45241314, -8.62910995], [0. , -0.78446454, -1.56892908, -2.35339362]]))</pre>
<pre>P,L,U = linalg.lu(a)</pre>	<pre>name 'linalg' is not defined; module 'numpy.linalg' has no attribute 'lu'</pre>
<pre>cg</pre>	<pre>name 'cg' is not defined</pre>

<code>np.fft.fft(a)</code>	<code>array([[10.+0.j, -2.+2.j, -2.+0.j, -2.-2.j], [26.+0.j, -2.+2.j, -2.+0.j, -2.-2.j]])</code>
<code>np.fft.ifft(a)</code>	<code>array([[2.5+0.j , -0.5-0.5j, -0.5+0.j , -0.5+0.5j], [6.5+0.j , -0.5-0.5j, -0.5+0.j , -0.5+0.5j]])</code>
<code>np.sort(a)</code>	<code>array([[1, 2, 3, 4], [5, 6, 7, 8]])</code>
<code>np.sort(a, axis=1)</code>	<code>array([[1, 2, 3, 4], [5, 6, 7, 8]])</code>
<code>I = np.argsort(a[:, 0]); b = a[I,:]</code>	In [127]: I Out[127]: array([0, 1]) In [128]: b Out[128]: array([[1, 2, 3, 4], [5, 6, 7, 8]])
<code>Z = np.array([[1, 2], [3, 4], [5, 6]]) y = np.array([7, 8, 9]) x = np.linalg.lstsq(Z, y) x</code>	<code>(array([-6. , 6.5]), array([3.19835777e-30]), 2, array([9.52551809, 0.51430058]))</code>
<code>x = np.array([9.52551809, 0.51430058]) q = 2 signal.resample(x, int(np.ceil(len(x)/2)))</code>	<code>array([5.01990934])</code>
<code>a = np.array([[1, 2, 3, 4], [5, 6, 7, 8]]) np.unique(x)</code>	<code>array([1, 2, 3, 4, 5, 6, 7, 8])</code>
<code>a = np.array([[1, 2, 3], [4, 5, 6]]) a.squeeze()</code>	<code>array([[1, 2, 3], [4, 5, 6]])</code>

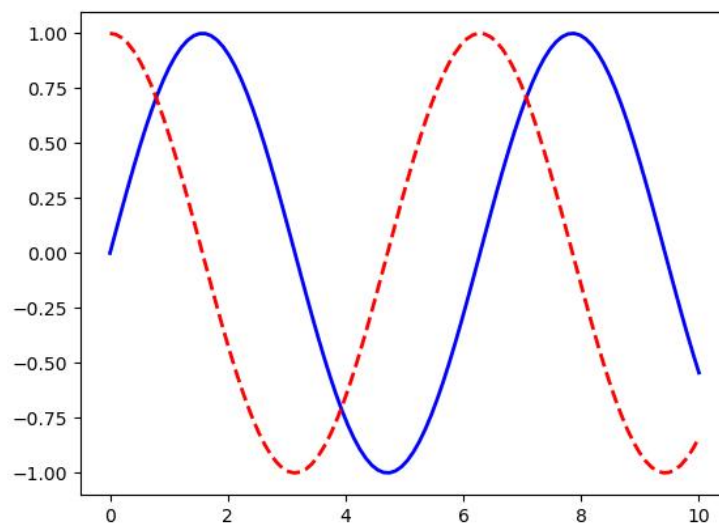
Task 3:

Task 4:

1. Code:

```
'''  
import matplotlib.pyplot as plt  
  
x = np.linspace(0, 10, 100)  
y1 = np.sin(x)  
y2 = np.cos(x)  
  
plt.plot(x, y1, label='Sine Wave', color='blue', linestyle='-', linewidth=2)  
plt.plot(x, y2, label='Cosine Wave', color='red', linestyle='--', linewidth=2)  
plt.show()  
'''
```

2. Figure:



Task 5:

I prefer to use Github.

Account: Bei129

Name: Lingyi Xu

Email: lingyi.xu129@gmail.com

Task 6:

<https://github.com/Bei129/Intro-to-Deep-Machine-Learning>